

FIFA World Cup 2026 Predictor League — Kiro Development Prompt

🏆 Project Overview

Build a **FIFA World Cup 2026 Predictor League** — a fantasy-league-style (FPL-inspired) responsive web application where users predict match outcomes and compete in league tables based on prediction accuracy. The app must work seamlessly on both **desktop and mobile** using **React Native for Web** (or React Native + Expo for cross-platform).

🌀 Core Concept

This is NOT a simple score predictor — it's a **full fantasy league experience** similar to FPL (Fantasy Premier League) but for the FIFA World Cup 2026. Users join leagues, make predictions, earn points, use strategic "Double Points" selections, and compete on live leaderboards.

🌐 Production Hosting — worldcupboys.world

Server Details:

Property	Value
Domain	worldcupboys.world
Home Directory	/home/worlnwxv
cPanel Username	worlnwxv
Disk Quota	20 GB (2.25 MB used)
Databases Available	0 / 50
SSL Certificate	⚠️ INACTIVE — Must activate via cPanel → SSL/TLS or Let's Encrypt
Server Theme	Jupiter
CPU Usage	23 / 200 (11.5%)
Bandwidth	Unlimited (∞)
FTP Accounts	0 / 50
Subdomains	0 / 30
Email Accounts	0 / 30
Physical Memory	1 GB

⚠️ Critical First Steps:

1. **Activate SSL** — Go to cPanel → SSL/TLS → Install Let's Encrypt for worldcupboys.world
2. **Create subdomain** — api.worldcupboys.world for backend API
3. **Enable Node.js** — cPanel → "Setup Node.js App"
4. **Set up FTP** — Create FTP account for deployment

🔒 Authentication & Landing Page

Google Single Sign-On (SSO)

- **Landing page IS the sign-in page** — modern, animated, World Cup 2026 themed
- Implement **Google OAuth 2.0** using @react-oauth/google or Firebase Auth
- Flow: Landing → Google SSO → Dashboard
- Store user profile (name, email, avatar) from Google account
- JWT token-based session management
- Refresh token rotation for security
- **Google OAuth redirect URI:** https://worldcupboys.world/auth/google/callback

Landing Page Requirements:

- Full-screen animated background with FIFA World Cup 2026 branding elements
- Animated trophy/football graphics (use Lottie animations)
- "Sign in with Google" button as primary CTA
- Tagline: "Predict. Compete. Win."
- Brief feature highlights with scroll animations
- Mobile-responsive hero section

🎨 FIFA World Cup 2026 Theme

Color Palette (Official WC 2026 inspired):

Primary: #1B0A3E (Deep Purple/Navy)
 Secondary: #E4002B (FIFA Red)
 Accent 1: #00B2A9 (Teal/Turquoise - Mexico)
 Accent 2: #002868 (USA Blue)
 Accent 3: #006847 (Mexico Green)
 Gold: #FFD700 (Trophy Gold)
 Background Dark: #0D0D1A
 Background Light: #F5F5F7

Design Elements:

- World Cup 2026 trophy silhouette in backgrounds
 - Stadium/pitch-inspired UI patterns
 - Country flag integrations
 - Animated football/soccer ball elements
 - Gradient overlays using the tri-nation theme (USA US, Mexico MX, Canada CA)
 - Card-based UI with glassmorphism effects
 - Micro-animations on interactions (Framer Motion / React Native Reanimated)
-

Tech Stack

Frontend

- **React Native** (with Expo for web + mobile)
- **React Native for Web** — single codebase for desktop & mobile
- **React Navigation** for routing
- **React Native Reanimated** for animations
- **Lottie** for complex animations
- **NativeWind** (TailwindCSS for React Native)
- **React Native Paper** or custom component library
- Dark & Light theme support (default: Dark theme)

Backend

- **Node.js** with **Express.js** or **NestJS**
- **GraphQL** (Apollo Server) + REST API fallback
- **Socket.io** for real-time leaderboard updates
- **Node-cron** for scheduled data fetching
- **Passport.js** for Google OAuth

Database

- **MongoDB Atlas** (Cloud)
 - Connection String: mongodb+srv://<db_username>:<db_password>@cluster0.xlslniry.mongodb.net/
 - ⚠️ **IMPORTANT: Store credentials in .env file, NEVER hardcode**
 - Database name: wc2026_predictor

Environment Variables (.env):

```
# Database
MONGODB_URI=mongodb+srv://<db_username>:<db_password>@cluster0.xlslniry.mongodb.net/wc2026_predictor

# Google OAuth
GOOGLE_CLIENT_ID=<your-google-client-id>
GOOGLE_CLIENT_SECRET=<your-google-client-secret>
GOOGLE_CALLBACK_URL=https://worldcupboys.world/auth/google/callback

# JWT
JWT_SECRET=<generate-strong-secret>
JWT_REFRESH_SECRET=<generate-another-strong-secret>

# Football API
FOOTBALL_API_KEY=<your-api-key>

# Server
PORT=5000
NODE_ENV=production
DOMAIN=worldcupboys.world

# Hosting
HOME_DIR=/home/worlwnxv
CPANEL_USER=worlwnxv
```

Add .env to .gitignore immediately.

Deployment & Hosting

Deployment Strategy for worldcupboys.world (cPanel):

Option A: Node.js on cPanel (Recommended)

1. **Enable Node.js** via cPanel → "Setup Node.js App"
2. Set Node.js version: **18.x or 20.x LTS**
3. Application root: `/home/worlwnxv/wc2026-predictor`
4. Application URL: `worldcupboys.world`
5. Application startup file: `server/server.js`
6. Run `npm install` via cPanel terminal
7. Set environment variables in cPanel Node.js app settings

Option B: Hybrid Deployment

- **Frontend (React Native Web build):** Deploy static build to `/home/worlwnxv/public_html/`
- **Backend API:** Run Node.js app on subdomain `api.worldcupboys.world`
- **Database:** MongoDB Atlas (cloud-hosted, already configured)

Option C: External Hosting (if cPanel Node.js is limited)

- **Frontend:** Vercel/Netlify with custom domain `worldcupboys.world` (point DNS)
- **Backend:** Railway or Render (free/hobby tier) at `api.worldcupboys.world`
- **Database:** MongoDB Atlas (already configured)

cPanel Setup Checklist:

```
# 1. SSH into server or use cPanel Terminal
cd /home/worlwnxv

# 2. Clone repository
git clone https://github.com/<your-username>/wc2026-predictor.git

# 3. Install dependencies
cd wc2026-predictor/server && npm install
cd ../client && npm install

# 4. Build frontend for production
npm run build # outputs to client/web-build/

# 5. Copy build to public_html (if serving static)
cp -r client/web-build/* /home/worlwnxv/public_html/

# 6. Configure .htaccess for React Router SPA
cat > /home/worlwnxv/public_html/.htaccess << 'EOF'
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /
  RewriteRule ^index\.html$ - [L]
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteRule . /index.html [L]
</IfModule>
EOF
```

cPanel Cron Jobs (Schedulers):

```
# Squad & Player Data – Every 6 hours
0 */6 * * * cd /home/worlwnxv/wc2026-predictor/server && /usr/bin/node schedulers/squadScheduler.js >> /home/worlwnxv/logs/squad.log 2>&1

# Fixture Data – Every hour
0 * * * * cd /home/worlwnxv/wc2026-predictor/server && /usr/bin/node schedulers/fixtureScheduler.js >> /home/worlwnxv/logs/fixture.log 2>&1

# Live Match Data – Every minute (during World Cup period only)
* * * * * cd /home/worlwnxv/wc2026-predictor/server && /usr/bin/node schedulers/liveMatchScheduler.js >> /home/worlwnxv/logs/live.log 2>&1

# Match Result Finalization – Every 5 minutes
*/5 * * * * cd /home/worlwnxv/wc2026-predictor/server && /usr/bin/node schedulers/resultScheduler.js >> /home/worlwnxv/logs/result.log 2>&1

# Prediction Lock – Every minute
* * * * * cd /home/worlwnxv/wc2026-predictor/server && /usr/bin/node schedulers/lockScheduler.js >> /home/worlwnxv/logs/lock.log 2>&1

# Leaderboard Recalculation – Every 10 minutes
*/10 * * * * cd /home/worlwnxv/wc2026-predictor/server && /usr/bin/node schedulers/leaderboardScheduler.js >> /home/worlwnxv/logs/leaderboard.log 2>&1

# Notification Dispatch – Every 15 minutes
*/15 * * * * cd /home/worlwnxv/wc2026-predictor/server && /usr/bin/node schedulers/notificationScheduler.js >> /home/worlwnxv/logs/notification.log 2>&1
```

SSL Setup (CRITICAL — Required for Google OAuth):

```
# Via cPanel → SSL/TLS → Let's Encrypt
# Or via cPanel → AutoSSL
# Domain: worldcupboys.world
# Also cover: www.worldcupboys.world, api.worldcupboys.world
```

DNS Configuration:

```
# A Record
worldcupboys.world → [Your Shared IP]
```

```
# CNAME (if using Vercel for frontend)
www.worldcupboys.world → cname.vercel-dns.com

# Subdomain for API
api.worldcupboys.world → [Your Shared IP] (or external service)
```

 MongoDB Schema Design

Collections:

```
// users
{
  _id: ObjectId,
  googleId: String,
  email: String,
  name: String,
  avatar: String,
  role: "player" | "admin",
  createdAt: Date,
  lastLogin: Date,
  preferences: {
    theme: "dark" | "light",
    notifications: { push: Boolean, email: Boolean, inApp: Boolean }
  }
}

// leagues
{
  _id: ObjectId,
  name: String,
  type: "public" | "private",
  inviteCode: String,
  inviteLink: String,
  ownerId: ObjectId (ref: users),
  members: [ObjectId],
  maxMembers: Number,
  createdAt: Date,
  settings: {
    scoringRules: Object,
    allowLateJoin: Boolean
  }
}

// leagueMembers
{
  _id: ObjectId,
  leagueId: ObjectId,
  userId: ObjectId,
  joinedAt: Date,
  status: "active" | "pending" | "removed",
  totalPoints: Number,
  rank: Number
}

// teams
{
  _id: ObjectId,
  name: String,
  code: String, // e.g., "ARG", "BRA"
  flag: String, // URL to flag image
  group: String, // e.g., "A", "B"
  fifaRanking: Number,
  squad: [ObjectId] // ref: players
}

// players
{
  _id: ObjectId,
  name: String,
  teamId: ObjectId,
  position: "GK" | "DEF" | "MID" | "FWD",
  shirtNumber: Number,
  isStarting: Boolean,
  photo: String,
  stats: {
    goals: Number,
    assists: Number,
    yellowCards: Number,
    redCards: Number
  }
}

// fixtures
{
  _id: ObjectId,
  matchday: Number,
  stage: "group" | "round_of_32" | "round_of_16" | "quarter_final" | "semi_final" | "third_place" | "final",
  homeTeam: ObjectId,
  awayTeam: ObjectId,
  kickoffTime: Date,
  venue: String,
  city: String,
  status: "scheduled" | "live" | "finished" | "postponed",
```

```
result: {
  homeScore: Number,
  awayScore: Number,
  firstTeamToScore: ObjectId | null,
  firstPlayerToScore: ObjectId | null
},
isLocked: Boolean,
lockTime: Date // kickoffTime - 10 minutes
}

// predictions
{
  _id: ObjectId,
  userId: ObjectId,
  fixtureId: ObjectId,
  homeScore: Number,
  awayScore: Number,
  firstTeamToScore: "home" | "away" | "none",
  firstPlayerToScore: ObjectId | null,
  isDoublePoints: Boolean,
  submittedAt: Date,
  lastEditedAt: Date,
  isLocked: Boolean,
  points: {
    scoreline: Number, // 0 or 3
    firstTeam: Number, // 0 or 5
    firstPlayer: Number, // 0 or 6
    doubleMultiplier: Boolean,
    total: Number
  }
}

// doublePointSelections
{
  _id: ObjectId,
  userId: ObjectId,
  matchday: Number,
  fixtureId: ObjectId,
  selectedAt: Date,
  isLocked: Boolean
}

// leaderboards
{
  _id: ObjectId,
  leagueId: ObjectId,
  matchday: Number,
  type: "matchday" | "weekly" | "overall",
  rankings: [{
    userId: ObjectId,
    rank: Number,
    previousRank: Number,
    movement: Number,
    totalPoints: Number,
    matchdayPoints: Number,
    correctScores: Number,
    correctFirstTeam: Number,
    correctFirstPlayer: Number,
    doublePointsSuccesses: Number
  }],
  publishedAt: Date
}

// matchResults
{
  _id: ObjectId,
  fixtureId: ObjectId,
  homeScore: Number,
  awayScore: Number,
  firstTeamToScore: ObjectId | null,
  firstPlayerToScore: ObjectId | null,
  goals: [{
    playerId: ObjectId,
    teamId: ObjectId,
    minute: Number,
    type: "goal" | "penalty" | "own_goal"
  }],
  isOfficial: Boolean,
  fetchedAt: Date,
  source: String
}

// notifications
{
  _id: ObjectId,
  userId: ObjectId,
  type: "fixture_reminder" | "deadline_reminder" | "leaderboard_published" | "rank_change" | "double_points_reminder",
  title: String,
  message: String,
  isRead: Boolean,
  createdAt: Date,
  metadata: Object
}
```

Live Data Integration & Schedulers

Football Data API

Use one of these APIs (in priority order):

1. **API-Football** (api-football.com) — Most comprehensive
2. **Football-Data.org** — Free tier available
3. **SportMonks** — Good real-time data

Scheduler Implementation (Node-Cron):

```
// schedulers/index.js

const cron = require('node-cron');

// 1. SQUAD & PLAYER DATA – Every 6 hours
cron.schedule('0 */6 * * *', async () => {
  await fetchAndUpdateSquads();
  await fetchAndUpdatePlayers();
  console.log('[SCHEDULER] Squad data updated');
});

// 2. FIXTURE DATA – Every hour
cron.schedule('0 * * * *', async () => {
  await fetchAndUpdateFixtures();
  console.log('[SCHEDULER] Fixtures updated');
});

// 3. LIVE MATCH DATA – Every 1 minute (during match windows)
cron.schedule('* * * * *', async () => {
  const liveMatches = await getLiveMatches();
  if (liveMatches.length > 0) {
    await fetchLiveScores(liveMatches);
    await fetchGoalScorers(liveMatches);
    console.log('[SCHEDULER] Live data updated');
  }
});

// 4. MATCH RESULT FINALIZATION – Every 5 minutes
cron.schedule('*/5 * * * *', async () => {
  await checkAndFinalizeResults();
  console.log('[SCHEDULER] Results checked');
});

// 5. PREDICTION LOCK – Every minute
cron.schedule('* * * * *', async () => {
  await lockExpiredPredictions();
  console.log('[SCHEDULER] Prediction locks checked');
});

// 6. LEADERBOARD RECALCULATION – Every 10 minutes
cron.schedule('*/10 * * * *', async () => {
  await recalculateLeaderboards();
  console.log('[SCHEDULER] Leaderboards recalculated');
});

// 7. NOTIFICATION DISPATCH – Every 15 minutes
cron.schedule('*/15 * * * *', async () => {
  await sendPendingNotifications();
  console.log('[SCHEDULER] Notifications dispatched');
});
```

Data Fetching Service:

```
// services/footballApi.js

const axios = require('axios');
const Team = require('../models/Team');
const Player = require('../models/Player');
const Fixture = require('../models/Fixture');
const MatchResult = require('../models/MatchResult');

const API_BASE = 'https://v3.football.api-sports.io';
const HEADERS = { 'x-apisports-key': process.env.FOOTBALL_API_KEY };
const WC_2026_LEAGUE_ID = 1; // Update with actual World Cup 2026 league ID

async function fetchAndUpdateSquads() {
  const response = await axios.get(`${API_BASE}/teams`, {
    headers: HEADERS,
    params: { league: WC_2026_LEAGUE_ID, season: 2026 }
  });

  for (const teamData of response.data.response) {
    await Team.findOneAndUpdate(
      { code: teamData.team.code },
      {
        name: teamData.team.name,
        code: teamData.team.code,
        flag: teamData.team.logo,
      },
    ),
  },
```

```

    { upsert: true }
  );

  const squadResponse = await axios.get(`${API_BASE}/players/squads`, {
    headers: HEADERS,
    params: { team: teamData.team.id }
  });

  for (const player of squadResponse.data.response[0].players) {
    await Player.findOneAndUpdate(
      { name: player.name, teamId: teamData.team.id },
      {
        name: player.name,
        position: mapPosition(player.position),
        shirtNumber: player.number,
        photo: player.photo,
      },
      { upsert: true }
    );
  }
}

async function fetchLiveScores(liveMatches) {
  for (const match of liveMatches) {
    const response = await axios.get(`${API_BASE}/fixtures`, {
      headers: HEADERS,
      params: { id: match.apiFixtureId }
    });

    const fixtureData = response.data.response[0];

    await Fixture.findByIdAndUpdate(match._id, {
      status: 'live',
      'result.homeScore': fixtureData.goals.home,
      'result.awayScore': fixtureData.goals.away,
    });

    io.emit('live-score-update', {
      fixtureId: match._id,
      homeScore: fixtureData.goals.home,
      awayScore: fixtureData.goals.away,
      minute: fixtureData.fixture.status.elapsed
    });
  }
}

async function fetchGoalScorers(liveMatches) {
  for (const match of liveMatches) {
    const response = await axios.get(`${API_BASE}/fixtures/events`, {
      headers: HEADERS,
      params: { fixture: match.apiFixtureId, type: 'Goal' }
    });

    const goals = response.data.response;
    if (goals.length > 0) {
      const firstGoal = goals[0];
      await MatchResult.findOneAndUpdate(
        { fixtureId: match._id },
        {
          firstTeamToScore: firstGoal.team.id,
          firstPlayerToScore: firstGoal.player.id,
          goals: goals.map(g => ({
            playerId: g.player.id,
            teamId: g.team.id,
            minute: g.time.elapsed,
            type: g.detail.toLowerCase().includes('penalty') ? 'penalty' :
              g.detail.toLowerCase().includes('own') ? 'own_goal' : 'goal'
          })))
        },
        { upsert: true }
      );
    }
  }
}

async function checkAndFinalizeResults() {
  const finishedMatches = await Fixture.find({
    status: 'finished',
    'result.isOfficial': { $ne: true }
  });

  for (const match of finishedMatches) {
    const result = await MatchResult.findOne({ fixtureId: match._id });
    if (result && result.goals.length >= 0) {
      await MatchResult.findByIdAndUpdate(result._id, { isOfficial: true });
      await calculateMatchPoints(match._id);
    }
  }
}

function mapPosition(apiPosition) {
  const map = { 'Goalkeeper': 'GK', 'Defender': 'DEF', 'Midfielder': 'MID', 'Attacker': 'FWD' };
  return map[apiPosition] || 'MID';
}

```

Automated Scoring Engine

```
// services/scoringEngine.js

async function calculateMatchPoints(fixtureId) {
  const fixture = await Fixture.findById(fixtureId);
  const result = await MatchResult.findOne({ fixtureId });
  const predictions = await Prediction.find({ fixtureId });

  for (const prediction of predictions) {
    let points = { scoreline: 0, firstTeam: 0, firstPlayer: 0, doubleMultiplier: false, total: 0 };

    // 1. Correct Scoreline (3 points)
    if (prediction.homeScore === result.homeScore &&
        prediction.awayScore === result.awayScore) {
      points.scoreline = 3;
    }

    // 2. First Team to Score (5 points)
    const actualFirstTeam = result.firstTeamToScore
      ? (result.firstTeamToScore.toString() === fixture.homeTeam.toString() ? 'home' : 'away')
      : 'none';
    if (prediction.firstTeamToScore === actualFirstTeam) {
      points.firstTeam = 5;
    }

    // 3. First Player to Score (6 points)
    if (prediction.firstPlayerToScore && result.firstPlayerToScore &&
        prediction.firstPlayerToScore.toString() === result.firstPlayerToScore.toString()) {
      points.firstPlayer = 6;
    }

    // 4. Calculate total
    points.total = points.scoreline + points.firstTeam + points.firstPlayer;

    // 5. Apply Double Points multiplier
    if (prediction.isDoublePoints) {
      points.total *= 2;
      points.doubleMultiplier = true;
    }

    // 6. Update prediction with points
    await Prediction.findByIdAndUpdate(prediction._id, { points, isLocked: true });

    // 7. Update user totals
    await updateUserTotalPoints(prediction.userId);
  }

  // 8. Recalculate all league leaderboards
  await recalculateAllLeaderboards(fixture.matchday);

  // 9. Emit real-time leaderboard update
  io.emit('leaderboard-updated', { fixtureId, matchday: fixture.matchday });

  // 10. Send notifications
  await sendRankChangeNotifications(fixture.matchday);
}
```

Application Pages & Routes

Public Routes:

1. / — Landing Page (Google SSO) → <https://worldcupboys.world/>
2. /about — About the app

Protected Routes (after auth):

1. /dashboard — User Dashboard (home)
2. /fixtures — All fixtures list
3. /fixtures/:id — Single fixture + prediction form
4. /predictions — My predictions history
5. /leaderboard — Global leaderboard
6. /leaderboard/:leagueId — League-specific leaderboard
7. /leagues — Browse/join leagues
8. /leagues/create — Create a league
9. /leagues/:id — League detail
10. /profile — User profile & settings
11. /notifications — Notification center
12. /analytics — Prediction analytics

Admin Routes:

1. /admin — Admin dashboard
2. /admin/fixtures — Manage fixtures
3. /admin/results — Update results
4. /admin/users — User management

- 5. `/admin/leagues` — League management
- 6. `/admin/scoring` — Scoring audit & rules

 Key Features (FPL-Style)

1. Matchday View

- Calendar-style fixture list grouped by matchday
- Countdown timers to prediction deadlines
- Live score badges on active matches
- Quick-predict cards (swipe on mobile)

2. Prediction Interface

- Score selector (+ / - buttons or swipe)
- Team selector for "First to Score"
- Player search/dropdown for "First Scorer"
- Double Points toggle (star icon) — only ONE per matchday
- Visual confirmation of locked predictions
- Points breakdown preview

3. Live Leaderboard

- Real-time rank updates via WebSocket
- Animated rank changes (green arrow up, red arrow down)
- Mini league tables
- Head-to-head comparison
- Matchday vs Overall toggle

4. User Dashboard (FPL-style)

- Points summary card (total, matchday, rank)
- Upcoming fixtures requiring predictions
- Double Points selection reminder
- Prediction accuracy donut chart
- Recent activity feed
- League standings snippet

5. Fantasy-Style Extras

- **Matchday Captain** = Double Points Match
- **Transfer Deadline** = Prediction Lock (10 min before KO)
- **Gameweek History** = Matchday History
- **Mini Leagues** = Private Leagues
- **Overall Rank** = Global Leaderboard

 Notification System

```
const NOTIFICATION_TYPES = {
  FIXTURE_REMINDER: '24h before kickoff – remind to submit predictions',
  DEADLINE_WARNING: '1h before prediction lock',
  DEADLINE_URGENT: '15min before prediction lock',
  MATCHDAY_RESULTS: 'After all matchday games finish',
  RANK_CHANGE: 'When user rank changes significantly (±3 positions)',
  DOUBLE_POINTS_REMINDER: 'If user hasn\'t selected DP match 2h before first KO',
  LEAGUE_INVITE: 'When invited to a league',
  NEW_MATCHDAY: 'When new matchday fixtures are available'
};

// Channels: Push (Firebase Cloud Messaging), Email (SendGrid/Nodemailer), In-App (Socket.io)
// Email can use: noreply@worldcupboys.world (set up via cPanel Email Accounts)
```

 GitHub Repository Structure

```
wc2026-predictor/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml
│   │   └── deploy.yml
├── client/
│   ├── app/
│   ├── components/
│   │   ├── common/
│   │   ├── predictions/
│   │   ├── leaderboard/
│   │   ├── fixtures/
│   │   └── animations/
```

```
├── hooks/
├── services/
├── store/
├── theme/
├── utils/
├── assets/
│   ├── animations/
│   ├── images/
│   └── fonts/
├── app.json
├── package.json
├── server/
│   ├── config/
│   │   ├── db.js
│   │   ├── passport.js
│   │   └── scheduler.js
│   ├── models/
│   ├── routes/
│   ├── controllers/
│   ├── services/
│   │   ├── footballApi.js
│   │   ├── scoringEngine.js
│   │   ├── leaderboardService.js
│   │   └── notificationService.js
│   ├── schedulers/
│   │   ├── squadScheduler.js
│   │   ├── fixtureScheduler.js
│   │   ├── liveMatchScheduler.js
│   │   ├── resultScheduler.js
│   │   └── lockScheduler.js
│   ├── middleware/
│   │   ├── auth.js
│   │   ├── rateLimiter.js
│   │   └── validation.js
│   ├── socket/
│   │   └── index.js
│   ├── .env
│   ├── server.js
│   └── package.json
├── shared/
│   ├── types/
│   └── constants/
├── .gitignore
├── .env.example
├── README.md
└── package.json
```

Implementation Priority

Phase 1 — MVP (Week 1-2)

1. ☒ Project setup (Expo + Node.js + MongoDB Atlas connection)
2. ☒ Google SSO authentication (Passport.js + JWT)
3. ☒ Landing page with animations at worldcupboys.world
4. ☒ User dashboard
5. ☒ Fixture listing from Football API
6. ☒ Basic prediction submission
7. ☒ Prediction lock mechanism (10 min before KO)
8. ☒ Deploy to cPanel / activate SSL

Phase 2 — Core Features (Week 3-4)

1. ☒ Scoring engine (auto-calculate on match finish)
2. ☒ Leaderboard system (overall + matchday + league)
3. ☒ Double Points feature (one per matchday, validated)
4. ☒ League creation & joining (public + private with invite codes)
5. ☒ Real-time updates (Socket.io for live scores + leaderboard)
6. ☒ All schedulers active via cPanel Cron Jobs

Phase 3 — Polish (Week 5-6)

1. ☒ Animations & transitions (page transitions, micro-interactions)
2. ☒ Notification system (push + email via worldcupboys.world + in-app)
3. ☒ Analytics dashboard (accuracy stats, trends)
4. ☒ Admin panel (fixture/result/user/league management)
5. ☒ Dark/Light theme toggle
6. ☒ Performance optimization (caching, lazy loading)
7. ☒ Testing & bug fixes

Phase 4 — Bonus Features

1. 🌟 Knockout stage bonus points
2. 🌟 Golden Boot prediction (top scorer of tournament)
3. 🌟 Tournament winner prediction
4. 🌟 Head-to-head challenges between users
5. 🌟 Achievement badges & XP system
6. 🌟 Social sharing (share prediction cards to Instagram/WhatsApp)

🏆

Scoring System Summary

Prediction Type	Points	Description
Correct Scoreline	3	Exact score match
First Team to Score	5	Correct team scores first
First Player to Score	6	Correct player scores first
Double Points Match x2	All points doubled for selected match	

Maximum per match: $(3 + 5 + 6) \times 2 = 28$ points

⚠️

Critical Rules & Constraints

1. **NEVER** **hardcode credentials** — use `.env` (add to `.gitignore`)
2. **Always lock predictions 10 minutes before kickoff** — server-side enforcement
3. **Only ONE Double Points match per matchday** — validate on both client AND server
4. **Real-time leaderboard** — use WebSocket (Socket.io), not HTTP polling
5. **Mobile-first design** — React Native ensures responsive by default
6. **SSL is MANDATORY** — Google OAuth requires HTTPS (<https://worldcupboys.world>)
7. **Rate limiting** — protect API endpoints (express-rate-limit)
8. **Data validation** — Joi or Zod schemas on all inputs (server-side)
9. **Error handling** — graceful degradation when football API is down
10. **Caching** — Redis or in-memory cache for leaderboard/fixture data
11. **Idempotent scoring** — prevent double-scoring if scheduler runs twice
12. **Timezone handling** — all times stored in UTC, converted on display
13. **Memory limit** — Server has 1GB RAM; optimize Node.js with `--max-old-space-size=768`

🎯

Success Metrics

- Support **1000+** **concurrent users** during peak match times
- Leaderboard updates within **5 seconds** of match events
- Prediction submission latency < **200ms**
- **99.9% uptime** during World Cup matches
- Mobile performance score > **90** (Lighthouse)
- Zero data loss on predictions
- API response time < **100ms** for cached endpoints
- Stay within **1GB memory** and **20GB disk** limits on cPanel

Generated for Kiro AI Development Assistant — FIFA World Cup 2026 Predictor League Domain: worldcupboys.world Last Updated: June 11, 2026